

How VPN tunnels work — a practical overview

Everything below is explained through the lens of Tailscale + WireGuard on a Synology NAS, but the core concepts apply to VPNs in general.

The problem VPNs solve

Your home NAS sits behind a router that uses NAT (Network Address Translation). NAT lets all your home devices share one public IP address, but it also means that unsolicited incoming traffic from the internet gets dropped — the router doesn't know which internal device to send it to.

Traditional solutions (port forwarding, DDNS) work by poking holes in this barrier, but they expose your NAS to the entire internet. A VPN creates a private, encrypted path between two specific devices without opening the door to everyone else.

Key concepts

NAT (Network Address Translation)

Every router between your device and the internet performs NAT. When your NAS sends a packet out, the router rewrites the source address from the NAS's internal IP (e.g., `192.168.1.100`) to the router's public IP (e.g., `203.0.113.50`) and assigns a random port (e.g., `41234`). It remembers this mapping so that when a reply comes back to `203.0.113.50:41234`, it knows to forward it to `192.168.1.100`.

The critical property: **outbound connections always work**. Return traffic for an outbound connection is allowed through. Inbound connections from strangers are blocked.

NAT hole punching

The technique that makes Tailscale (and many other peer-to-peer systems) work without port forwarding.

1. Device A sends a packet outbound to Device B's public IP:port.
2. Device B simultaneously sends a packet outbound to Device A's public IP:port.
3. Both routers see outbound traffic and create NAT mappings.
4. Both routers now accept incoming packets from the other's public IP:port, because they look like return traffic.

Result: a direct bidirectional connection through both NATs, with no ports explicitly opened.

Tunneling and encapsulation

A VPN "tunnel" isn't a physical path — it's a logical one. Your original data packet (e.g., an HTTP request to Synology Photos) gets encrypted, then wrapped inside a new UDP packet with normal internet routing headers. Routers along the way see only the outer UDP packet and route it normally. They have no idea there's an encrypted payload inside.

At the other end, the VPN software strips the outer headers, decrypts the payload, and delivers the original packet as if both devices were on the same local network.

WireGuard

The VPN protocol used by Tailscale. Key properties:

- **~4,000 lines of code** (vs 100,000+ for OpenVPN), making it far easier to audit for security.
- **Uses modern cryptography:** Curve25519 for key exchange, ChaCha20 for encryption, Poly1305 for authentication.
- **Runs in the kernel** on most platforms, so it's fast and battery-efficient.
- **Stateless design:** if a connection drops, it resumes immediately when packets flow again — no complex reconnection handshake.

Persistent keepalives

NAT mappings expire after a period of inactivity (typically 30–120 seconds for UDP). To prevent the tunnel from breaking, WireGuard sends a tiny empty encrypted packet every 25 seconds. This keeps the NAT mapping alive on both sides.

Cost: roughly 100 bytes per packet = ~350 KB/day. Negligible for battery and bandwidth.

How Tailscale establishes a connection

Phase 1 — Registration

Both devices make **outbound** connections to Tailscale's coordination server. Each device reports:

- Its WireGuard public key
- The public IP:port assigned by its router's NAT
- Its Tailscale identity (authenticated via Google, Microsoft, GitHub, etc.)

The coordination server acts as an **address book** — it learns where each device can be reached, but never receives any WireGuard private keys.

Phase 2 — NAT hole punching

When your phone wants to reach your NAS:

1. The coordination server tells the phone: "Your NAS is at `203.0.113.50:41234`."
2. It tells the NAS: "Your phone is at `198.51.100.80:52918`."
3. Both devices simultaneously send UDP packets to each other's public IP:port.
4. Both routers create allow rules for the return traffic.
5. The "holes" are punched — direct bidirectional UDP flow is established.

Phase 3 — Direct tunnel

WireGuard takes over the now-open UDP path and encrypts everything end-to-end. The actual packet route is:

Phone → LTE tower → internet backbone → your ISP → your router → NAS

Tailscale's servers are completely out of this path. The coordination server's job is done.

Fallback — DERP relay

If hole punching fails (rare — happens with symmetric NAT on both sides), Tailscale routes traffic through its DERP (Designated Encrypted Relay for Packets) relay servers. Even then:

- The data is still **WireGuard-encrypted end-to-end**.
- The relay sees only opaque encrypted bytes.
- The relay **cannot decrypt** the traffic (it has no WireGuard keys).
- Performance is slower due to the extra hop.

What happens when the network changes

Event	What happens
Phone switches Wi-Fi → LTE	Public IP changes. Keepalives fail. Tailscale re-coordinates and re-punches within 1–3 seconds.
Router reboots	NAT table clears. Same recovery — re-punch from both sides.
NAS reboots	Tailscale boot-up script runs, NAS re-registers, hole punch resumes.
Internet goes down briefly	WireGuard is stateless — once connectivity returns, it resumes instantly without a handshake.

Security analysis

What's protected

- **Data in transit:** WireGuard encryption (ChaCha20-Poly1305) — no one between your phone and NAS can read the payload, including your ISP, carrier, and coffee shop Wi-Fi.
- **Against coordination server compromise:** Even if an attacker gains control of Tailscale's servers, they cannot decrypt existing traffic because they don't have the WireGuard private keys. Your private keys never leave your devices.
- **Against man-in-the-middle:** WireGuard authenticates both ends with public/private key pairs. An attacker can't insert themselves between your phone and NAS without having the correct private key.

What's NOT protected

- **Endpoint compromise:** If your phone or NAS is infected with malware, the attacker is inside the tunnel. The VPN only protects the pipe, not the devices at either end. Mitigation: strong passwords, 2FA on DSM, keep DSM updated, don't install untrusted packages.
- **Tailscale identity provider compromise:** If an attacker gains access to your Google/Microsoft account (used to log in to Tailscale), they could authorize a new device onto your tailnet. Mitigation: 2FA on your identity provider, enable device approval in Tailscale admin.
- **Tailscale trust model:** You trust Tailscale to honestly broker key exchanges and not inject malicious peers. They can't read your data, but they can influence which devices join your network. Mitigation: use Tailscale ACLs to restrict access, or self-host with Headscale (open-source coordination server replacement).
- **Cryptographic advances:** WireGuard's Curve25519 key exchange could be weakened by future quantum computing breakthroughs. Not a practical concern today, but worth noting for long-term perspective.

Defense in depth — layers of security in our setup

Layer 1: Tailscale identity (Google/Microsoft SSO + device approval)
Layer 2: WireGuard encryption (Curve25519 + ChaCha20 + Poly1305)
Layer 3: DSM authentication (username + password + optional 2FA)
Layer 4: Synology Photos permissions (per-user access controls)
Layer 5: No open ports on router (zero public attack surface)

An attacker would need to breach multiple independent layers to access your photos.

Glossary

Term	Meaning
NAT	Network Address Translation — how routers let many devices share one public IP
Hole punching	Technique where two devices behind NAT simultaneously send packets to each other, tricking both routers into allowing the traffic
WireGuard	Modern, minimal VPN protocol (~4K lines of code, kernel-level, very fast)
Tailscale	Service that uses WireGuard + NAT hole punching + coordination to create zero-config VPN meshes
Tailnet	Your personal Tailscale network — all your devices form a mesh
DERP	Designated Encrypted Relay for Packets — Tailscale's fallback relay when hole punching fails
Keepalive	Small periodic packet sent to prevent NAT mappings from expiring
Coordination server	Tailscale's address book — knows where devices are, but can't read their traffic
Headscale	Open-source, self-hosted replacement for Tailscale's coordination server
E2E encryption	End-to-end encryption — only the two endpoint devices can decrypt the data